

Sessions and Locks

Seeing who is logged in, ending a session that will not end itself, and inspecting or forcing the locks a session holds.

These are the verbs for looking at the machine as a whole and intervening in it: **whose sessions are running, what they are holding, and how to take either away**. Every one of them is administrator-tier, and the ones that act on somebody else need an elevated session on top of that.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a user-set page is worth naming, it is named in words.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0. The lock listings come from a program that took the locks first and then ran the verb against itself, because a lock nobody holds is not something a listing can show.

Two things are not on this page, on purpose

A programmer needs to understand locks without being able to force one, so the *user* documentation keeps the part of the subject that is theirs: what a task lock is against a database lock, and `release`, which gives back locks your own session holds and needs no privilege at all. The same goes for `status`, `phantom`, `pstat`, `pdebug` and `pdump` — a programmer's view of their own processes.

This page is the other half: the verbs that see and act across the machine.

Who is logged in: `listu`

```
listu {no.page} {lptr {n}}
```

```
:listu
```

User	Pid	Puid	Login time	Origin	Username
------	-----	------	------------	--------	----------

```
12 1789          27 Aug 13:14 : don
19 1808          27 Aug 13:53 : don
* 27 1828        27 Aug 13:59 : don
```

***** the session you are typing in

User SD's user number — what `logout`, `pstat` and `pdump` all take

Pid the Windows process id

Puid the user number of the **parent**, filled in for a phantom and blank otherwise

Origin where the session came from

Username the **Windows** account the session runs as, not the SD account it is in

Username is the column that matters for permission. Almost every control on this page is keyed to the Windows identity, not to the SD account — so two sessions showing different accounts but the same username are, as far as `logout` is concerned, the same person's.

Origin is blank for a local session and that is not a fault. It reports an address or a device name, and a console or piped session has neither. It reads `Phantom` for a phantom and an IP address for a network client.

`SDNet` is also among the values it can print, and you will not see it: SDNet was removed from this port, so nothing can open the kind of session that would report it.

`(logout pending)` after the name means somebody has asked that session to end and it has not gone. See below.

Ending a session: `logout`

```
logout          end this session
logout n {n ...} end another session, by user number
logout all      end every session but this one
```

`logout` with no argument ends your own session. It is `quit` under another name, which is worth knowing before typing it intending to list something.

```
:logout 999
Only administrators can logout processes running with other usernames
```

A session may end only sessions running under the same Windows account unless it is elevated. The refusal above came from an unelevated session, and user 999 does not exist — **the privilege test happens before the number is looked up**, so this message does not tell you whether the user was real.

`logout all` is stricter still: elevated **and** run from the `SDSYS` account. It leaves your own session alone.

WHEN A SESSION WILL NOT END

`logout` **signals** a process. If the process has already gone — killed from outside SD, or lost with a terminal — there is nothing to signal, and the entry stays with `(logout pending)` beside it.

The entry matters because it holds a slot and an exclusive-access claim. `NUMUSERS` counts it, and any verb that wants a file to itself — `build.index` is the usual one — is refused while it is there. **Recovery is not another `logout`:**

```
sd -cleanup
```

run elevated, and a restart of the SD service if that does not take it.

Confirm the session is actually dead before clearing it. `pstat` *n* answers (*Not responding*) for a session with nothing behind it — it asks the process and waits about four seconds — and that is the difference between a dead entry and a busy one. `pstat` is a programmer verb and is documented in the user set.

What is locked: `list.readu`

```
list.readu {user.no} {detail} {wait} {no.page} {lptr {n}}
```

```
:list.readu
```

```
There are no active file, read or update locks held by any user
```

That answer is about the **whole system**, not about you. With locks held:

User	File	Path.....	Type	Id.....
23	1	/cygdrive/c/ProgramData/SD/user	RU	R1
		_accounts/don/ZZLK31A		
19	4	/cygdrive/c/ProgramData/SD/user	RU	zzlock31
		_accounts/don/voc		
23	1	/cygdrive/c/ProgramData/SD/user	RL	R2
		_accounts/don/ZZLK31A		

User	the SD user number holding it — <code>listu</code> turns that into a person
File	SD's internal file number, and this is the number <code>unlock</code> wants
Path	wrapped over as many lines as it needs, in POSIX form
Type	<code>RU</code> update · <code>RL</code> read · <code>FX</code> exclusive file lock · <code>SX</code> shared file lock · <code>WAIT</code> a session waiting for one
Id	the record id; blank for a file lock

A file lock has no id:

```
User File Path..... Type Id.....
 23    1 /cygdrive/c/ProgramData/SD/user FX
      _accounts/don/ZZLK31A
```

The PATH is the POSIX one and that is not a display fault. SD holds file paths internally in `/cygdrive/c/...` form on this port. It names the same place as `C:\ProgramData\SD\...`.

THE KEYWORDS

`detail` prints the record-lock budget above the listing:

```
:list.readu detail
Record lock limit (NUMLOCKS) = 100, Current = 0, Peak = 2
There are no active file, read or update locks held by any user
```

Peak is the one to watch. `Current` describes this instant, which is rarely the instant that mattered. A peak near the limit is how you find out `NUMLOCKS` needs raising **before** a program fails with the lock table full. `config` reports the configured limit.

A user number restricts the listing to one session. `wait` adds the sessions *waiting* for a lock, as `WAIT` rows — left out by default, so the plain listing shows the cause of a hold-up and not its victims. **Ask for `wait` when something is stuck.**

A LOCK OUTLIVING ITS SESSION

The `19 ... RU zzlock31` row above is real, and it is a lock on a record in an account's `voc` file left behind by a session that was killed while holding it. **A dead session's locks are not released**, any more than its user-table entry is, and everything wanting that record waits for a process that is not there.

Take the user number to `pstat`. (*Not responding*) means clear the session rather than wait for it.

Task locks: `list.locks`, `lock`, `clear.locks`

Task locks are **64 numbered flags with no connection to any data**. They mean whatever the programs using them agree they mean — usually *only one session runs this job at a time*, where the thing being protected is not a single file.

```
list.locks
lock n {no.wait}
clear.locks {n}
```

```
:list.locks
No task locks reserved by any user
```

and with lock 5 held by user 16, `*` marking your own session:

```
:list.locks
0:      1:      2:      3:      4:      5: 16*    6:      7:
8:      9:     10:     11:     12:     13:     14:     15:
16:     17:     18:     19:     20:     21:     22:     23:
```

(64 numbers over eight rows; the rest are blank and are cut here.)

`LOCK` WAITS UNLESS YOU TELL IT NOT TO

```
:lock 5 no.wait
Set task lock 5
:lock 5 no.wait
Task lock already owned by this process
```

Without `no.wait` it waits for ever. It prints *Waiting for task lock to become available* once, then retries every two seconds until it gets it — right for a job that must run, wrong for anything unattended. `no.wait` **turns the wait into a refusal** (*Task lock is already in use*), which a script can act on.

`CLEAR.LOCKS` GIVES BACK YOUR OWN, AND ONLY YOUR OWN

```
:clear.locks 5
Released task lock 5
:clear.locks
All task locks released
```

With no number it releases all 64 this session holds, and says *all* whether it held any or not.

<i>Released task lock n</i>	it was yours and it is now free
-----------------------------	---------------------------------

<i>Task lock n is held by another process</i>	<code>clear.locks</code> will not take it
---	--

<i>Task lock n is not held by any process</i>	it was already free
---	---------------------

<i>Task lock number must be in range 0 to 63</i>	from <code>clear.locks 99</code>
--	----------------------------------

Forcing a lock open: `unlock`

```
unlock file n {user n} record.id {record.id ...}
unlock file n {user n} all
unlock file n {user n} filelock
unlock tasklock n {n ...}
```

This is the only verb that takes somebody else's lock, and it needs an elevated session — having the verb is not enough:

```
:unlock
Command requires administrator privileges
```

It names a file by number, not by name — the `File` column of `list.readu`. That is deliberate: the lock is on a file the machine has open, which may not be in your VOC at all and may be reached under different names in different accounts.

<code>file</code> <i>n</i>	the file, by the number <code>list.readu</code> printed
----------------------------	---

<code>user</code> <i>n</i>	restrict to one session's locks
----------------------------	---------------------------------

<code>all</code>	every record lock matching, rather than named ids
------------------	---

<code>filelock</code>	the file lock, which cannot be combined with record ids
-----------------------	---

<code>tasklock</code> <i>n</i>	force a task lock, including one held by another process
--------------------------------	--

A file number or a user number is compulsory — *Either a file number or a user number must be specified* — so there is no `unlock` that means *everything on the machine*.

Unlocking is not free and SD cannot make it so. A lock is a promise its holder is relying on. Forcing one open while its owner is alive lets two sessions write the same record and tells neither. **Establish that the holder is dead first** — `pstat` on the user number, `listu` for (*logout pending*) — and prefer clearing the session to clearing the lock.

`UNLOCK TASKLOCK` IS THE ONLY WAY BACK FROM A KILLED HOLDER

Task locks are released when a session ends normally. They are not part of any file, so nothing has to be written back.

`sd -cleanup` **does not give them back, and that is a defect.** It releases a dead session's record locks and file locks and leaves its task locks held, by a user number nothing is behind, until SD itself is restarted. `list.locks` shows the number with an owner and `clear.locks` refuses it because it is not yours. `unlock tasklock n`, **elevated, is the way out** — that is what the forced form is for. It is recorded in the project's fix lists.

Who has these verbs

All of them are administrator-tier, so a standard or programmer account does not have the names at all.

the verb is enough

`listu` `list.readu` `list.locks` `lock` `clear.locks`

needs elevation as well

`unlock`, and `logout` against another Windows account's session

The split is between looking and intervening. Seeing who is on the machine and what they hold is an administrator's ordinary business. Taking a session or a lock away from somebody else asks for the elevated token as well.

See also

Accounts and Security · Operating System Access.